

Lab 1

PingDirectory Setup, Configuration & Verification

Devanshu Trivedi

IAM Security Analyst

Environment: Ping Identity DevOps Docker Lab - PingDirectory · PingFederate · PingAccess

August 2026

Table of Contents

1. Objective	3
2. Lab Environment Verification	3
3. Building the CyberCorp Directory Information Tree.....	6
4. Directory Snapshot.....	21
5. Securing the PingFederate Service Account	24
6. Troubleshooting & Lessons Learned.....	26
7. Summary	28

1. Objective

This lab documents the verification and directory information tree (DIT) configuration of a PingDirectory instance running in a Docker-based Ping Identity DevOps environment alongside PingFederate and PingAccess. The goal is to stand up a representative directory structure for a fictitious organization, CyberCorp (dc=cybercorp,dc=local), consisting of organizational units for people, groups, and service accounts; a sample user identity; a sample security group; and a dedicated service account that PingFederate can use to perform LDAP-based user lookups.

Every configuration change is followed by an independent verification step using `ldapsearch`, and access control is configured so that the PingFederate service account is granted read access to the People branch of the directory. The final sections of this document capture the issues encountered during the exercise and the reasoning used to resolve each one.

2. Lab Environment Verification

Before making any configuration changes, the running Docker environment was inspected to confirm that the PingDirectory container — and the accompanying PingFederate and PingAccess containers — were healthy and reachable.

Component	Detail
Docker Engine	29.7.2 (Community), API 1.55
PingDirectory image	pingidentity/pingdirectory:edge (11.1.0.0-alpine_3.24.1-al21)
Other containers	pingfederate (9031/9999), pingaccess (3000/9000) – both healthy
PingDirectory ports	1389 (LDAP), 8443→1443 (HTTPS admin)
Target naming context	dc=cybercorp,dc=local

```
devanshu@ping-lab:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CR
EATED	STATUS	PORTS	NAMES
62745ddd4574	pingidentity/pingaccess	"/bootstrap.sh star..."	4
days ago	Up 3 days (healthy)	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 1443/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	pingaccess
85ade2b05e0a	pingidentity/pingfederate:latest	"/bootstrap.sh star..."	4
days ago	Up 3 days (healthy)	0.0.0.0:9031->9031/tcp, [::]:9031->9031/tcp, 0.0.0.0:9999->9999/tcp, [::]:9999->9999/tcp	pingfedera
te			
db519e0d416f	pingidentity/pingdirectory:edge	"/bootstrap.sh star..."	4
days ago	Up 3 days (healthy)	1636/tcp, 0.0.0.0:1389->1389/tcp, [::]:1389->1389/tcp, 1689/tcp, 0.0.0.0:8443->1443/tcp, [::]:8443->1443/tcp	pingdirect
ory			

Figure 1. Confirming the pingdirectory, pingfederate, and pingaccess containers are up and healthy (docker ps).

```
devanshu@ping-lab:~$ docker images
```

IMAGE	ID	DISK USAGE	Info →	U	In Use
hello-world:latest	5dd0d3e6e255	25.9kB	CONTENT SIZE	9.49kB	EXTRA
pingidentity/pingaccess:latest	fb4fb7ec7fa6	858MB			U
pingidentity/pingdirectory:edge	055ff46fe4eb	678MB			U
pingidentity/pingdirectory:latest	055ff46fe4eb	678MB			U
pingidentity/pingfederate:latest	82aeda6ddd26	1.53GB			U

Figure 2. Docker images pulled for the Ping Identity DevOps lab stack (docker images).

```
devanshu@ping-lab:~$ docker version
Client: Docker Engine - Community
Version: 29.7.2
API version: 1.55
Go version: go1.26.5
Git commit: a7dcaa6
Built: Wed Aug 5 18:28:53 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.7.2
API version: 1.55 (minimum version 1.40)
Go version: go1.26.5
Git commit: 6a43e3d
Built: Wed Aug 5 18:28:53 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.3.3
GitCommit: aad11006b869517fcd3009450b6f82da282e1a9b
runc:
Version: 1.4.3
GitCommit: v1.4.3-0-gbb14dabe
docker-init:
Version: 0.19.0
GitCommit: de40ad0
devanshu@ping-lab:~$
```

Figure 3. Docker Engine client and server version information (docker version).

```
devanshu@ping-lab:~$ docker inspect pingdirectory
[
  {
    "Id": "db519e0d416fa9920c894045f9e735035e81661e09baf7649f5c8e72894e3c
ea",
    "Created": "2026-08-21T02:34:52.474826032Z",
    "Path": "./bootstrap.sh",
    "Args": [
      "start-server"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 5646,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-08-23T01:53:50.832795206Z",
      "FinishedAt": "2026-08-22T06:28:20.666873657Z",
      "Health": {
        "Status": "healthy",
        "FailingStreak": 0,
        "Log": [

```

Figure 4. Runtime and health details for the pingdirectory container (docker inspect pingdirectory).

```
devanshu@ping-lab:~$ docker exec -it pingdirectory sh

#####
###
                Ping Identity DevOps Docker Image

    Product: PingDirectory
    Version: 11.1.0.0-alpine_3.24.1-al21
    DevOps User: dtrived4@charlotte.edu
    Hostname: db519e0d416f
    Started: 20260823
#####
###

                Security Warning

The server profile being used to configure this Ping Identity Docker Image
is for demo and documentation purposes only. The configuration contains default
keys/credentials etc and is not suitable for production as it carries a substantial
security risk.

For additional information, please see
https://github.com/pingidentity/pingidentity-devops-getting-started/blob/master/SECURITY.md

#####
###
```

Figure 5. Interactive shell session inside the PingDirectory container, showing the product banner and the built-in security warning for demo profiles (docker exec).

```

> ls -la /opt/out/instance
total 152
drwxr-xr-x 22 ping root 4096 Aug 21 02:35 .
drwxrwx--- 1 ping root 4096 Aug 21 02:35 ..
-rw-r--r-- 1 ping root 303 Aug 21 02:35 PingDirectory.lic
-rw-r----- 1 ping root 527 Aug 21 02:34 README
-rw-r----- 1 ping root 244 Aug 21 02:34 README.md
drwxr-x--- 2 ping root 4096 Aug 21 02:34 bak
drwxr-x--- 2 ping root 4096 Aug 21 02:34 bat
drwxr-x--- 2 ping root 4096 Aug 21 02:34 bin
-rw-r----- 1 ping root 1021 Aug 21 02:34 build-info.txt
drwx----- 2 ping root 4096 Aug 21 02:35 collector
drwxr-x--- 11 ping root 4096 Aug 26 01:05 config
drwxr-x--- 2 ping root 4096 Aug 21 02:34 csd-files
drwxr-x--- 4 ping root 4096 Aug 21 02:35 db
drwxr-x--- 5 ping root 4096 Aug 21 02:34 docs
drwx----- 2 ping root 4096 Aug 21 02:35 history
drwxr-x--- 2 ping root 4096 Aug 21 02:35 import-tmp
drwxr-x--- 2 ping root 4096 Aug 26 01:05 ldif
drwxr-x--- 3 ping root 4096 Aug 21 02:34 legal
drwxr-x--- 5 ping root 20480 Aug 21 02:35 lib
drwxr-x--- 2 ping root 4096 Aug 21 02:35 locks
drwxr-x--- 9 ping root 4096 Aug 26 01:51 logs
drwxr-x--- 2 ping root 4096 Aug 21 02:34 metrics
drwxr-x--- 2 ping root 4096 Aug 21 02:34 profile-files
drwxr-x--- 5 ping root 4096 Aug 21 02:34 resource
-rwxr-x--- 1 ping root 2427 Aug 21 02:34 revert-update
-rwxr-x--- 1 ping root 2776 Aug 21 02:34 setup
drwxr-x--- 2 ping root 4096 Aug 21 02:34 tmp
-rwxr-x--- 1 ping root 3393 Aug 21 02:34 update
drwxr-x--- 4 ping root 4096 Aug 21 02:34 velocity
-rw-r----- 1 ping root 17970 Aug 21 02:34 vendor_logo.png

```

Figure 6. Contents of the PingDirectory installation directory (/opt/out/instance).

3. Building the CyberCorp Directory Information Tree

With the environment confirmed healthy, the CyberCorp directory tree was built up incrementally: a dedicated backend, a base entry, three organizational units, a sample user, a sample group, and a service account. Each step below shows the exact command issued, the reasoning behind it, and the verification that followed.

3.1 Creating the CyberCorp Backend

```

./dsconfig --no-prompt create-backend \
  --backend-name cybercorp \
  --type local-db \
  --set enabled:true \
  --set base-dn:dc=cybercorp,dc=local

```

Reason: this creates the storage/configuration boundary that will serve dc=cybercorp,dc=local.

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./dsconfig --no-prompt create-backend \
> --backend-name cybercorp \
> --type local-db \
> --set enabled:true \
> --set base-dn:dc=cybercorp,dc=local

Arguments from tool properties file: --useSSL --hostname localhost --port 1636 --bindDN cn=administrator
--bindPasswordFile /opt/staging/.sec/root-user-password --trustAll

The Local DB Backend was created successfully
PingDirectory:db519e0d416f:/opt/out/instance/bin
>
```

Figure 7. Output of dsconfig create-backend confirming the cybercorp backend was created.

3.2 Verifying the Backend

```
./dsconfig get-backend-prop \
--backend-name cybercorp
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./dsconfig get-backend-prop \
> --backend-name cybercorp

Arguments from tool properties file: --useSSL --hostname localhost --port 1636 --bindDN cn=administrator
--bindPasswordFile /opt/staging/.sec/root-user-password --trustAll

Property                                : Value(s)
-----
backend-id                               : cybercorp
base-dn                                  : "dc=cybercorp,dc=local"
composite-index-entry-limit              : 100000
db-cache-percent                          : 10
db-directory                             : db
db-import-cache-percent                  : 60
description                              : -
enabled                                  : true
hash-entries                             : false
index-entry-limit                        : 4000
prime-method                             : none
PingDirectory:db519e0d416f:/opt/out/instance/bin
>
```

Figure 8. dsconfig get-backend-prop output confirming the cybercorp backend is enabled with base DN dc=cybercorp,dc=local.

3.3 Creating the CyberCorp Base Entry

Objective: create dc=cybercorp,dc=local with the following attributes:

- objectClass: top
- objectClass: domain
- dc: cybercorp

```
./ldapmodify --noPropertiesFile \  
  --hostname localhost \  
  --port 1389 \  
  --bindDN "cn=administrator" \  
  --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'  
dn: dc=cybercorp,dc=local  
changetype: add  
objectClass: top  
objectClass: domain  
dc: cybercorp  
EOF
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin  
> ./ldapmodify --noPropertiesFile \  
>   --hostname localhost \  
>   --port 1389 \  
>   --bindDN "cn=administrator" \  
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'  
> dn: dc=cybercorp,dc=local  
> changetype: add  
> objectClass: top  
> objectClass: domain  
> dc: cybercorp  
> EOF  
# Successfully connected to localhost:1389.  
  
# Adding entry dc=cybercorp,dc=local ...  
# Result Code: 0 (success)  
  
PingDirectory:db519e0d416f:/opt/out/instance/bin  
>
```

Figure 9. ldapmodify output confirming the dc=cybercorp,dc=local base entry was added successfully.

3.4 Verifying the Base Entry

```
./ldapsearch --noPropertiesFile \  
  --hostname localhost \  
  --port 1389 \  
  --bindDN "cn=administrator" \  
  --bindPasswordFile /opt/staging/.sec/root-user-password \  
  --baseDN "dc=cybercorp,dc=local" \  
  --scope base \  
  "(objectClass=*)" \  
  "dn" "objectClass" "dc"
```



```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "dc=cybercorp,dc=local" \
> --scope base \
> "(objectClass=*)" \
> "dn" "objectClass" "dc"
dn: dc=cybercorp,dc=local
dc: cybercorp
objectClass: top
objectClass: domain

# Result Code: 0 (success)
# Number of Entries Returned: 1

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |
```

Figure 10. ldapsearch output verifying the dc=cybercorp,dc=local base entry.

3.5 Creating the People Organizational Unit

```
./ldapmodify --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: ou=People,dc=cybercorp,dc=local
changetype: add
objectClass: top
objectClass: organizationalUnit
ou: People
EOF
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: ou=People,dc=cybercorp,dc=local
> changetype: add
> objectClass: top
> objectClass: organizationalUnit
> ou: People
> EOF
# Successfully connected to localhost:1389.

# Adding entry ou=People,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

PingDirectory:db519e0d416f:/opt/out/instance/bin
>
```

Figure 11. *ldapmodify output confirming the ou=People organizational unit was created.*

Verify:

```
./ldapsearch --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password \
--baseDN "ou=People,dc=cybercorp,dc=local" \
--scope base \
"(objectClass=*)" \
"dn" "objectClass" "ou"
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "ou=People,dc=cybercorp,dc=local" \
> --scope base \
> "(objectClass=*)" \
> "dn" "objectClass" "ou"
dn: ou=People,dc=cybercorp,dc=local
ou: People
objectClass: top
objectClass: organizationalUnit

# Result Code: 0 (success)
# Number of Entries Returned: 1

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |
```

Figure 12. *ldapsearch output verifying the ou=People organizational unit.*

3.6 Creating the Groups Organizational Unit

```
./ldapmodify --noPropertiesFile \  
  --hostname localhost \  
  --port 1389 \  
  --bindDN "cn=administrator" \  
  --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'  
dn: ou=Groups,dc=cybercorp,dc=local  
changetype: add  
objectClass: top  
objectClass: organizationalUnit  
ou: Groups  
EOF
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin  
> ./ldapmodify --noPropertiesFile \  
>   --hostname localhost \  
>   --port 1389 \  
>   --bindDN "cn=administrator" \  
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'  
> dn: ou=Groups,dc=cybercorp,dc=local  
> changetype: add  
> objectClass: top  
> objectClass: organizationalUnit  
> ou: Groups  
> EOF  
# Successfully connected to localhost:1389.  
  
# Adding entry ou=Groups,dc=cybercorp,dc=local ...  
# Result Code: 0 (success)  
  
PingDirectory:db519e0d416f:/opt/out/instance/bin  
>
```

Figure 13. ldapmodify output confirming the ou=Groups organizational unit was created.

Verify:

```
./ldapssearch --noPropertiesFile \  
  --hostname localhost \  
  --port 1389 \  
  --bindDN "cn=administrator" \  
  --bindPasswordFile /opt/staging/.sec/root-user-password \  
  --baseDN "ou=Groups,dc=cybercorp,dc=local" \  
  --scope base \  
  "(objectClass=*)" \  
  "dn" "objectClass" "ou"
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password \
>   --baseDN "ou=Groups,dc=cybercorp,dc=local" \
>   --scope base \
>   "(objectClass=*)" \
>   "dn" "objectClass" "ou"
dn: ou=Groups,dc=cybercorp,dc=local
ou: Groups
objectClass: top
objectClass: organizationalUnit

# Result Code: 0 (success)
# Number of Entries Returned: 1

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |
```

Figure 14. ldapsearch output verifying the ou=Groups organizational unit.

3.7 Creating the ServiceAccounts Organizational Unit

```
./ldapmodify --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "cn=administrator" \
  --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: ou=ServiceAccounts,dc=cybercorp,dc=local
changetype: add
objectClass: top
objectClass: organizationalUnit
ou: ServiceAccounts
EOF
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: ou=ServiceAccounts,dc=cybercorp,dc=local
> changetype: add
> objectClass: top
> objectClass: organizationalUnit
> ou: ServiceAccounts
> EOF
# Successfully connected to localhost:1389.

# Adding entry ou=ServiceAccounts,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |
```

Figure 15. ldapmodify output confirming the ou=ServiceAccounts organizational unit was created.

Verify:

```
./ldapsearch --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password \
--baseDN "ou=ServiceAccounts,dc=cybercorp,dc=local" \
--scope base \
"(objectClass=*)" \
"dn" "objectClass" "ou"
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "ou=ServiceAccounts,dc=cybercorp,dc=local" \
> --scope base \
> "(objectClass=*)" \
> "dn" "objectClass" "ou"

dn: ou=ServiceAccounts,dc=cybercorp,dc=local
ou: ServiceAccounts
objectClass: top
objectClass: organizationalUnit

# Result Code: 0 (success)
# Number of Entries Returned: 1
```

Figure 16. ldapsearch output verifying the ou=ServiceAccounts organizational unit.

3.8 Creating a Sample User — Alice Smith

```
./ldapmodify --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
changetype: add
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: Alice.Smith
cn: Alice Smith
sn: Smith
givenName: Alice
displayName: Alice Smith
mail: Alice.Smith@cybercorp.local
employeeNumber: 10001
departmentNumber: IT
title: Security Engineer
EOF
```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password \
>   --baseDN "ou=People,dc=cybercorp,dc=local" \
>   --scope sub \
>   "(uid=Alice.Smith)" \
>   "*"
# Result Code: 0 (success)
# Number of Entries Returned: 0

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
> changetype: add
> objectClass: top
> objectClass: person
> objectClass: organizationalPerson
> objectClass: inetOrgPerson
> uid: Alice.Smith
> cn: Alice Smith
> sn: Smith
> givenName: Alice
> displayName: Alice Smith
> mail: Alice.Smith@cybercorp.local
> employeeNumber: 10001
> departmentNumber: IT
> title: Security Engineer
> EOF
# Successfully connected to localhost:1389.

# Adding entry uid=Alice.Smith,ou=People,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |

```

Figure 17. Initial search for uid=Alice.Smith returning zero entries, followed by ldapmodify successfully adding the entry.

Verify:

```

./ldapsearch --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "cn=administrator" \
  --bindPasswordFile /opt/staging/.sec/root-user-password \
  --baseDN "ou=People,dc=cybercorp,dc=local" \
  --scope sub \
  "(uid=Alice.Smith)" \
  "*"

```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "ou=People,dc=cybercorp,dc=local" \
> --scope sub \
> "(uid=Alice.Smith)" \
> "*"
dn: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
mail: Alice.Smith@cybercorp.local
sn: Smith
cn: Alice Smith
departmentNumber: IT
givenName: Alice
employeeNumber: 10001
title: Security Engineer
uid: Alice.Smith
displayName: Alice Smith

# Result Code: 0 (success)
# Number of Entries Returned: 1
```

Figure 18. ldapsearch output verifying Alice Smith's directory entry and attributes.

3.9 Creating a Sample Group — IT-Users

```
./ldapmodify --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: cn=IT-Users,ou=Groups,dc=cybercorp,dc=local
changetype: add
objectClass: top
objectClass: groupOfNames
cn: IT-Users
member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
EOF
```



```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: cn=IT-Users,ou=Groups,dc=cybercorp,dc=local
> changetype: add
> objectClass: top
> objectClass: groupOfNames
> cn: IT-Users
> member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
> EOF
# Successfully connected to localhost:1389.

# Adding entry cn=IT-Users,ou=Groups,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=administrator" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: cn=Security-Users,ou=Groups,dc=cybercorp,dc=local
> changetype: add
> objectClass: top
> objectClass: groupOfNames
> cn: Security-Users
> member: uid=Bob.Jones,ou=People,dc=cybercorp,dc=local
> EOF
# Successfully connected to localhost:1389.

# Adding entry cn=Security-Users,ou=Groups,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

```

Figure 19. ldapmodify output confirming the IT-Users group was created under ou=Groups.

Verify:

```

./ldapsearch --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "cn=administrator" \
  --bindPasswordFile /opt/staging/.sec/root-user-password \
  --baseDN "ou=Groups,dc=cybercorp,dc=local" \
  --scope sub \
  "(objectClass=groupOfNames)" \
  "dn" "cn" "member"

```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "ou=Groups,dc=cybercorp,dc=local" \
> --scope sub \
> "(objectClass=groupOfNames)" \
> "dn" "cn" "member"

dn: cn=IT-Users,ou=Groups,dc=cybercorp,dc=local
cn: IT-Users
member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local

dn: cn=Security-Users,ou=Groups,dc=cybercorp,dc=local
cn: Security-Users
member: uid=Bob.Jones,ou=People,dc=cybercorp,dc=local

# Result Code: 0 (success)
# Number of Entries Returned: 2

PingDirectory:db519e0d416f:/opt/out/instance/bin
>
PingDirectory:db519e0d416f:/opt/out/instance/bin
>

```

Figure 20. ldapsearch output verifying the groupOfNames entries under ou=Groups.

3.10 Creating the PingFederate Service Account

```

./ldapmodify --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local
changetype: add
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
uid: svc-pingfederate
cn: PingFederate Service Account
sn: ServiceAccount
givenName: PingFederate
displayName: PingFederate Service Account
description: Service account used by PingFederate for directory integration
EOF

```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local
son
objectClass:> changetype: add
> objectClass: top
> objectClass: person
> objectClass: organizationalPerson
> objectClass: inetOrgPerson
> uid: svc-pingfederate
> cn: PingFederate Service Account
> sn: ServiceAccount
> givenName: PingFederate
> displayName: PingFederate Service Account
> description: Service account used by PingFederate for directory integration
> EOF
# Successfully connected to localhost:1389.

# Adding entry uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local
...
# Result Code: 0 (success)

```

Figure 21. *ldapmodify output confirming the svc-pingfederate service account was created.*

Verify:

```

./ldapsearch --noPropertiesFile \
--hostname localhost \
--port 1389 \
--bindDN "cn=administrator" \
--bindPasswordFile /opt/staging/.sec/root-user-password \
--baseDN "ou=ServiceAccounts,dc=cybercorp,dc=local" \
--scope sub \
"(objectClass=person)" \
"dn" "uid" "cn" "description"

```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "ou=ServiceAccounts,dc=cybercorp,dc=local" \
> --scope sub \
> "(objectClass=person)" \
> "dn" "uid" "cn" "description"
dn: uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local
cn: PingFederate Service Account
description: Service account used by PingFederate for directory integration
uid: svc-pingfederate

# Result Code: 0 (success)
# Number of Entries Returned: 1

```

Figure 22. *ldapsearch output verifying the svc-pingfederate service account entry.*

4. Directory Snapshot

With the base entry, organizational units, sample identity, sample group, and service account all in place, a full subtree search against `dc=cybercorp,dc=local` captures the tree as it stood at this point in the lab.

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --baseDN "dc=cybercorp,dc=local" \
> --scope sub \
> "(objectClass=*)" \
> "dn" "objectClass" "uid" "cn" "ou" "member"
dn: dc=cybercorp,dc=local
objectClass: top
objectClass: domain

dn: ou=People,dc=cybercorp,dc=local
ou: People
objectClass: top
objectClass: organizationalUnit

dn: ou=Groups,dc=cybercorp,dc=local
ou: Groups
objectClass: top
objectClass: organizationalUnit

dn: ou=ServiceAccounts,dc=cybercorp,dc=local
ou: ServiceAccounts
objectClass: top
objectClass: organizationalUnit

dn: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
cn: Alice Smith
uid: Alice.Smith
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: uid=Bob.Johnson,ou=People,dc=cybercorp,dc=local
cn: Bob Johnson
uid: Bob.Johnson
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: cn=IT-Users,ou=Groups,dc=cybercorp,dc=local
cn: IT-Users
member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

dn: cn=Security-Users,ou=Groups,dc=cybercorp,dc=local
cn: Security-Users
member: uid=Bob.Johnson,ou=People,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames
```

Figure 23. Full directory tree (part 1 of 2) — base entry, organizational units, the Alice Smith and Bob Johnson user entries, and the IT-Users / Security-Users groups.

```

dn: cn=IT-Admins,ou=Groups,dc=cybercorp,dc=local
cn: IT-Admins
member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

dn: cn=Security-Admins,ou=Groups,dc=cybercorp,dc=local
cn: Security-Admins
member: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
member: uid=Bob.Johnson,ou=People,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

dn: uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local
cn: PingFederate Service Account
uid: svc-pingfederate
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: uid=svc-app01,ou=ServiceAccounts,dc=cybercorp,dc=local
cn: Application Service Account
uid: svc-app01
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: uid=svc-database,ou=ServiceAccounts,dc=cybercorp,dc=local
cn: Database Service Account
uid: svc-database
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: uid=svc-monitoring,ou=ServiceAccounts,dc=cybercorp,dc=local
cn: Monitoring Service Account
uid: svc-monitoring
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson

dn: cn=App-ServiceAccounts,ou=Groups,dc=cybercorp,dc=local
cn: App-ServiceAccounts
member: uid=svc-app01,ou=ServiceAccounts,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

dn: cn=Database-ServiceAccounts,ou=Groups,dc=cybercorp,dc=local
cn: Database-ServiceAccounts
member: uid=svc-database,ou=ServiceAccounts,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

dn: cn=Monitoring-ServiceAccounts,ou=Groups,dc=cybercorp,dc=local
cn: Monitoring-ServiceAccounts
member: uid=svc-monitoring,ou=ServiceAccounts,dc=cybercorp,dc=local
objectClass: top
objectClass: groupOfNames

```

Figure 24. Full directory tree (part 2 of 2) — the IT-Admins and Security-Admins groups, the svc-pingfederate service account, and the additional application/database/monitoring service accounts and their associated groups.

5. Securing the PingFederate Service Account

5.1 Setting the Service Account Password

```
./ldappasswordmodify --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "cn=administrator" \
  --bindPasswordFile /opt/staging/.sec/root-user-password \
  --authzID "uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local"
```

```
PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldappasswordmodify --noPropertiesFile \
> --hostname localhost \
> --port 1389 \
> --bindDN "cn=administrator" \
> --bindPasswordFile /opt/staging/.sec/root-user-password \
> --authzID "uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local"

# Password Modify Extended Result:
# Result Code: 0 (success)
# Password Modify Extended Result Generated Password: ShoveSleevedRulingAesthetics

Successfully changed the password using a password modify extended operation.

The server generated a new password of 'ShoveSleevedRulingAesthetics'.
PingDirectory:db519e0d416f:/opt/out/instance/bin
>
```

Figure 25. ldappasswordmodify output showing the server-generated password for svc-pingfederate.

Note: The password shown above (ShoveSleevedRulingAesthetics) was auto generated by PingDirectory for this local, throwaway lab instance. As a general practice before publishing lab artifacts to a public repository, rotate or redact any generated credentials, bind DN's, and file paths that resemble production conventions.

5.2 Granting Read Access via an Access Control Instruction (ACI)

Giving the service account read access:

```
./ldapmodify --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "cn=administrator" \
  --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
dn: ou=People,dc=cybercorp,dc=local
changetype: modify
add: aci
aci: (targetattr="*)(version 3.0; aci "Cascade PingFederate read People";
  allow (read,search,compare)
  userdn="ldap:///uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local";)
EOF
```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapmodify --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "cn=admin" \
>   --bindPasswordFile /opt/staging/.sec/root-user-password <<'EOF'
> dn: ou=People,dc=cybercorp,dc=local
> changetype: modify
> add: aci
> aci: (targetattr="*)(version 3.0; acl "Cascade PingFederate read People"; allow (
read,search,compare) userdn="ldap://
/uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local";)
> EOF
# Successfully connected to localhost:1389.

# Modifying entry ou=People,dc=cybercorp,dc=local ...
# Result Code: 0 (success)

PingDirectory:db519e0d416f:/opt/out/instance/bin
> |

```

Figure 26. *ldapmodify output confirming the read-access ACI was added to ou=People.*

5.3 Validating End-to-End Access

```

./ldapsearch --noPropertiesFile \
  --hostname localhost \
  --port 1389 \
  --bindDN "uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local" \
  --bindPassword "ShoveSleevedRulingAesthetics" \
  --baseDN "ou=People,dc=cybercorp,dc=local" \
  --scope sub \
  "(uid=Alice.Smith)" \
  "uid" "cn" "mail" "departmentNumber" "title"

```

```

PingDirectory:db519e0d416f:/opt/out/instance/bin
> ./ldapsearch --noPropertiesFile \
>   --hostname localhost \
>   --port 1389 \
>   --bindDN "uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local" \
>   --bindPassword "ShoveSleevedRulingAesthetics" \
>   --baseDN "ou=People,dc=cybercorp,dc=local" \
>   --scope sub \
>   "(uid=Alice.Smith)" \
>   "uid" "cn" "mail" "departmentNumber" "title"
dn: uid=Alice.Smith,ou=People,dc=cybercorp,dc=local
mail: Alice.Smith@cybercorp.local
cn: Alice Smith
departmentNumber: IT
title: Security Engineer
uid: Alice.Smith

# Result Code: 0 (success)
# Number of Entries Returned: 1

```

Figure 27. *ldapsearch output confirming svc-pingfederate can now authenticate and read Alice Smith's attributes.*

6. Troubleshooting & Lessons Learned

Eleven distinct issues were diagnosed over the course of this lab. Each is captured below as the symptom encountered, the underlying cause, and the resolution or lesson that followed - forming a reusable checklist for future PingDirectory work.

Issue 1 — LDAP change record vs. LDIF import syntax

Running `ldapmodify` against an entry that omitted the `changetype` line produced the error “The second line of the change record ... does not begin with either 'control:' or 'changetype:'.”

`ldapmodify` processes LDAP change records, which require an explicit `changetype: add` line. An import-only LDIF file (used for bulk loads) can contain the bare entry without `changetype: add`, but `ldapmodify` cannot.

Lesson: remember the distinction - `ldapmodify` needs a `changetype` line; import LDIF does not.

Issue 2 — “Result Code: 0” with zero entries returned

After the base entry was added successfully, the follow-up `ldapsearch` reported “Result Code: 0 (success)” but “Number of Entries Returned: 0,” which looked contradictory at first glance.

It isn't. Result Code 0 only means the LDAP operation itself completed without error - it does not mean the search matched anything. In other words: the search worked, but nothing matching the filter and scope was returned.

Lesson: always check “Number of Entries Returned” separately from the result code when validating a search.

Issue 3 — Confirming the backend was actually configured

`dsconfig get-backend-prop` was used to confirm the backend directly: `cybercorp : local-db : true : “dc=cybercorp,dc=local”`, with `base-dn “dc=cybercorp,dc=local”`, `enabled: true`, and `writability-mode: enabled`.

This ruled out two possible causes of the empty search: that the CyberCorp backend didn't exist, and that the backend was disabled.

Issue 4 — Root DSE naming-context check

Querying the Root DSE (`baseDN ""`, `scope base`) for `namingContexts` returned both `dc=example,dc=com` and `dc=cybercorp,dc=local`.

This proved PingDirectory knew about the CyberCorp naming context, ruling out a registration problem as the cause of the empty search results.

```
./ldapsearch ... \
--baseDN "" \
--scope base \
"(objectClass=*)" \
"namingContexts"
```


Issue 5 — Anonymous vs. authenticated search behavior

The original verification search omitted `--bindDN` and `--bindPasswordFile`, making it an effectively anonymous search — while the entry itself had been created using the administrator account.

PingDirectory access controls can restrict what an anonymous client is allowed to see, so the search was re-run bound as `cn=administrator`, which succeeded.

This separates two concepts: authentication (“who are you?” - `cn=administrator`) from authorization / ACI (“what are you allowed to see?” - `dc=cybercorp,dc=local`).

Issue 6 — Schema-disallowed attribute: department vs. departmentNumber

Attempting to set `department: IT` on Alice's entry failed because that attribute was not permitted by the object classes configured on the entry (`person` / `organizationalPerson` / `inetOrgPerson`).

The problem wasn't that the attribute name was meaningless — it was that the entry's object classes did not permit it. Checking the schema confirmed `departmentNumber` was available instead, so `departmentNumber: IT` was used.

Lesson: LDAP attributes are governed by the directory schema and the entry's object classes — you can't add an attribute just because it makes logical sense.

Issue 7 — svc-pingfederate could bind but not find Alice

The service account `uid=svc-pingfederate,ou=ServiceAccounts,dc=cybercorp,dc=local` successfully authenticated (LDAP bind succeeded), but a subsequent search for Alice under `ou=People` returned zero entries.

Issue 8 — Comparing administrator vs. service-account searches

Running the identical search as `cn=administrator` returned Alice's entry in full, while the same search bound as `svc-pingfederate` returned nothing.

This ruled out Alice not existing, a wrong base DN, a wrong search filter, and a broken backend - and pointed squarely at an authorization / access-control problem specific to the service account.

Issue 9 — Querying the wrong dsconfig configuration component

`dsconfig get-global-configuration-prop --property global-aci` failed with “The property 'global-aci' is not a recognized property of Global Configurations.”

The mistake was targeting the wrong configuration component: `global-aci` belongs to the Access Control Handler, not the Global Configuration. The correct command was `dsconfig get-access-control-handler-prop --property global-aci`, which returned the configured global ACIs.

Lesson: if `dsconfig` reports a property as unrecognized, verify the configuration component being queried before assuming the property itself doesn't exist.

Issue 10 — Overly complex ACI failed with “invalid attribute syntax”

The first attempt to grant svc-pingfederate read access listed a long set of target attributes (uid, cn, sn, givenName, displayName, mail, departmentNumber, title) and failed with Result Code 21 (invalid attribute syntax).

Simplifying the ACI to targetattr="*" as a troubleshooting baseline succeeded, confirming that ACI processing itself was working and that access could in fact be granted to the service account. The attribute list can be narrowed again once the end-to-end flow is confirmed working.

Lesson: don't change multiple parts of a complex access-control rule at once - start from a broad, working baseline and tighten scope from there.

Issue 11 — Final validation

With the corrected ACI in place, svc-pingfederate successfully bound to PingDirectory and retrieved Alice Smith's uid, cn, mail, departmentNumber, and title — completing directory-side validation for the PingFederate integration.

7. Summary

By the end of this lab, the CyberCorp backend was fully provisioned with a base entry; People, Groups, and ServiceAccounts organizational units; a sample identity (Alice Smith); a sample security group (IT-Users); and a dedicated svc-pingfederate service account. The service account was assigned a directory-generated password and granted read, search, and compare access to the People branch through an access control instruction, and that access was validated end-to-end by binding as svc-pingfederate and successfully retrieving Alice Smith's attributes.

Along the way, eleven distinct issues were diagnosed and resolved — spanning LDIF change-record syntax, the difference between operation success and empty search results, backend and naming-context verification, anonymous vs. authenticated search behavior, schema-permitted attributes, dsconfig configuration-component scoping, and access-control-instruction troubleshooting. Together they form a reusable checklist for future PingDirectory configuration and integration work.